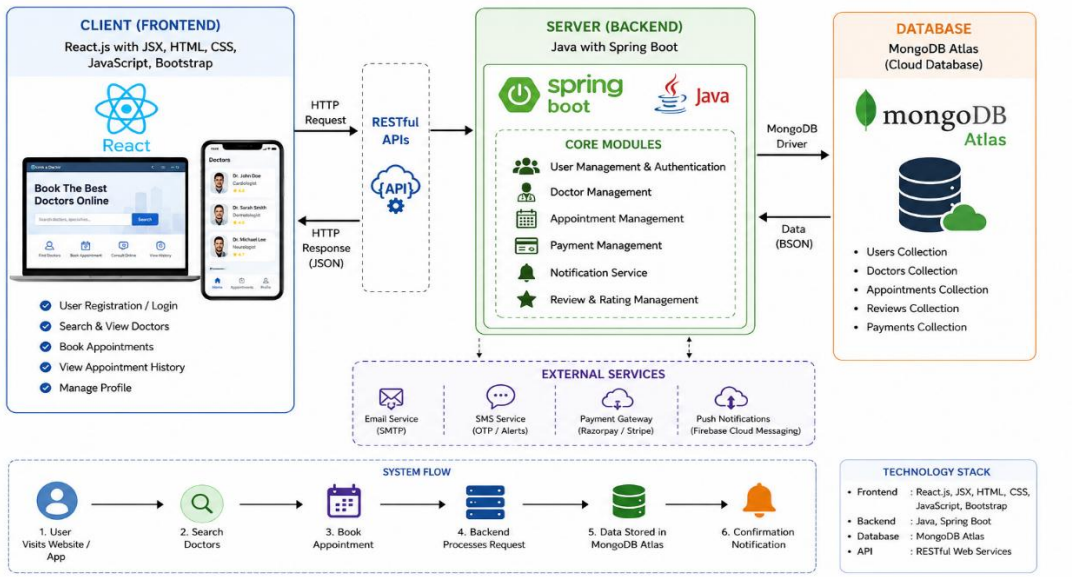
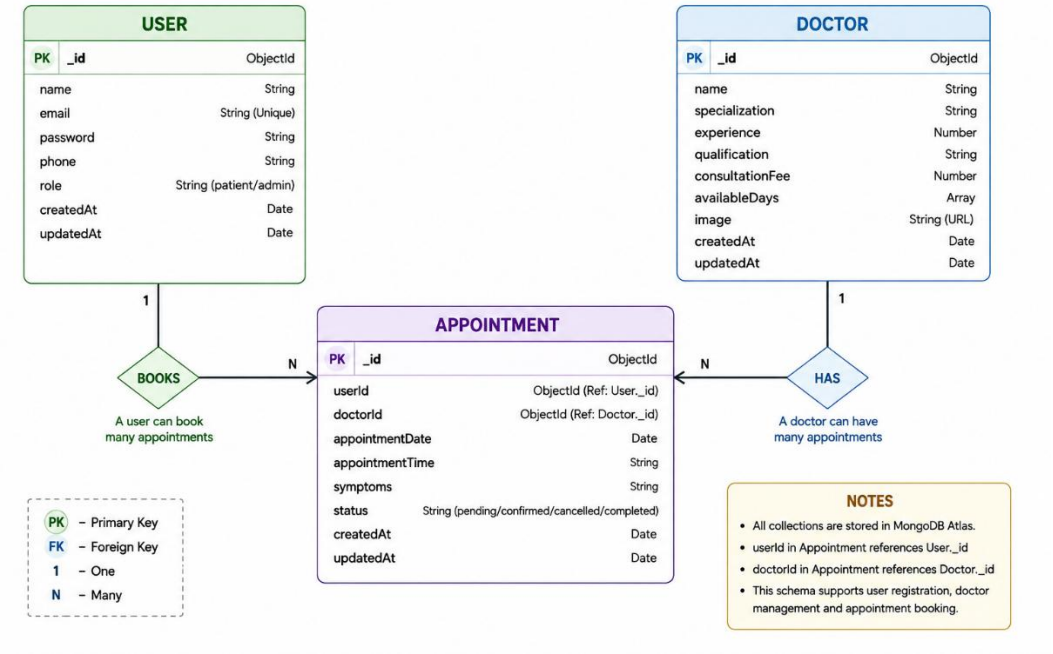
TECHNICAL ARCHITECTURE



The Technical Architecture of our Book a Doctor System follows a **client-server architecture**, where the **React frontend** acts as the client and the **Spring Boot backend** acts as the server. The frontend provides a modern, responsive, and user-friendly interface that enables patients to register, log in, search for doctors, book appointments, view appointment history, and manage their profiles. It is developed using **React.js, JSX, HTML, CSS, and JavaScript**, which together create reusable and interactive user interface components. The frontend communicates with the backend through **RESTful APIs** to exchange data efficiently. The backend is developed using **Java with Spring Boot**, which handles the application logic, user authentication, doctor management, appointment scheduling, and data processing. The system uses **MongoDB Atlas**, a cloud-based NoSQL database, to securely store patient information, doctor details, appointment records, and user credentials. Together, the **React.js frontend, JSX components, Spring Boot backend, REST APIs, and MongoDB Atlas database** provide a secure, scalable, and efficient platform for patients to book appointments online while enabling doctors and administrators to manage schedules, appointments, and patient records effectively.

ER DIAGRAM



This ER diagram represents the Book a Doctor System. It illustrates the relationships between the **Users, Doctors, and Appointments** in the system. Users can register, log in, search for doctors based on their specialization, view doctor profiles, and book appointments by providing the required details. Each appointment is associated with a specific user and doctor, enabling efficient appointment scheduling and management. Doctors can manage their profiles, availability, and appointment schedules, while users can view their booking history and appointment status. The system securely stores all user, doctor, and appointment information in **MongoDB Atlas**, ensuring reliable data storage, efficient retrieval, and scalability. Overall, the ER diagram provides a clear understanding of the database structure and the relationships among the main entities, illustrating the flow of information throughout the **Book a Doctor** application and supporting efficient appointment booking and healthcare management.

FEATURES

1. User Registration & Authentication

- **User Registration:** New users can create an account by providing their name, email address, phone number, and password to access the Book a Doctor system.
- **Secure Login:** Registered users can securely log in using their email and password to access personalized healthcare services and manage their appointments.
- **Profile Management:** Users can view and update their personal information, including contact details, to maintain accurate records.
- **User Registration:** New users can create an account by providing their name, email address, phone number, and password to access the Book a Doctor system.

2. Doctor Search & Profiles

- **Doctor Search:** Users can search for doctors based on their specialization and browse the list of available doctors.
- **Doctor Profiles:** The system displays each doctor's profile, including their specialization, qualifications, experience, consultation fee, and availability, helping users choose the right doctor.

3. Appointment Booking

- **Online Appointment Booking:** Users can book appointments by selecting a doctor and providing the required appointment details.
- **Appointment Management:** Users can view their booked appointments, appointment history, and current appointment status from their dashboard.

4. Doctor Management

- **Doctor Profile Management:** Doctors can maintain and update their profile information, including specialization, availability, consultation fee, and other professional details.
- **Appointment Handling:** Doctors can view scheduled appointments and manage their consultation schedule efficiently.

5. System Management

- **User and Doctor Data Management:** The system securely manages patient and doctor information, ensuring authorized access and efficient data processing.
- **Database Management:** All user, doctor, and appointment information is securely stored in **MongoDB Atlas**, providing reliable, scalable, and efficient cloud-based data storage.

6. Appointment Status & Communication

- **Appointment Status Tracking:** Users can check the status of their appointments, including booked, confirmed, completed, or cancelled appointments.
- **Appointment Updates:** The system keeps users informed whenever appointment details are updated, ensuring smooth communication and effective appointment management.

ROLES AND RESPONSIBILITIES

1. User Registration

Example:

Priya wants to consult a doctor for her health concern. She visits the **Book a Doctor** application and creates an account by entering her name, email address, phone number, and password. After successful registration, she logs into the system.

2. Browsing Doctors & Selecting a Specialty

Example:

Priya browses the list of available doctors and searches based on medical specialization such as General Physician, Dermatology, Cardiology, or Pediatrics. She selects a suitable doctor after viewing the doctor's specialization, experience, availability, and other profile details.

3. Booking an Appointment

Example:

Priya selects her preferred doctor and chooses an available date and time slot. She provides the required appointment details and confirms the booking through the system. The appointment is stored in the database and linked to her account.

4. Appointment Confirmation & Notification

Example:

After successful booking, the system generates an appointment confirmation containing the doctor's name, appointment date, time, and booking details. Priya receives a notification confirming that her appointment has been successfully scheduled.

5. Appointment Status Tracking

Example:

Using her dashboard, Priya can view her upcoming and previous appointments. She can monitor the appointment status, such as **Pending, Confirmed, Completed, or Cancelled**, and view relevant updates provided by the doctor.

6. Admin Verification & Management

Example:

The system administrator manages user and doctor accounts and verifies doctor information before allowing doctors to provide services through the platform. The administrator also manages departments, doctor profiles, appointments, and access permissions to maintain the security and reliability of the system.

7. Doctor Appointment Handling

Example:

The doctor logs into the system and views the list of scheduled appointments. The doctor reviews patient appointment details, accepts or manages the appointment, and updates the appointment status after consultation.

8. **Document Upload & Consultation**

Example:
Before or during the appointment, Priya can securely upload relevant medical documents or reports through the application. The doctor can access the required documents to better understand the patient's medical information and provide appropriate consultation.

9. **Appointment Completion & Notification**

Example:
After the consultation is completed, the doctor updates the appointment status to **Completed**. The system records the updated appointment information and sends a notification to Priya confirming that the consultation has been completed.

10. **System Monitoring**

Example:
The administrator monitors user registrations, doctor activities, appointments, document management, and system operations. The administrator can manage records, monitor appointment activities, generate reports, and ensure the smooth and secure functioning of the **Book a Doctor** platform.

USER FLOW

User (Patient)

- **Role:** Register on the platform, search for doctors, book appointments, manage medical documents, track appointments, and manage account details.
- **Flow:**

1. Account Creation and Login

    - Register using name, email, phone number, and password.
    - Log in securely to access the Book a Doctor platform.

2. Doctor Search
    - Browse available doctors based on specialization and department.
    - View doctor profiles, including specialization, experience, availability, and other relevant information.
3. Appointment Booking
    - Select a preferred doctor and available date and time slot.
    - Enter the required appointment details.
    - Confirm the appointment and receive a unique appointment confirmation.
4. Appointment Monitoring
    - View upcoming and previous appointments from the patient dashboard.
    - Track appointment status such as Pending, Confirmed, Completed, and Cancelled.
5. Medical Document Management
    - Upload relevant medical reports, prescriptions, or supporting documents.
    - View and manage previously uploaded documents securely.
6. Notifications and Updates
    - Receive notifications regarding appointment confirmation, cancellation, or updates.
    - View important information related to scheduled consultations.

7. Profile Maintenance

    - Update personal information and contact details.
    - View appointment history and previous consultations.

Doctor

- Role: Manage doctor profile, handle appointments, review patient information, access medical documents, and update consultation status.
- Flow:

    1. Registration and Login

        - Create a doctor account by providing the required professional and personal information.
        - Log in using authorized credentials.

    2. Profile Management

        - Create and maintain a professional profile.
        - Add details such as specialization, qualifications, experience, availability, and consultation information.

    3. Appointment Management

        - Access scheduled appointments through the doctor dashboard.
        - View patient appointment details and manage available time slots.

    4. Patient Information and Documents

        - Review relevant patient information associated with appointments.
        - Access medical reports and documents uploaded by the patient when required for consultation.

5. Consultation Management

    - Conduct the scheduled consultation.
    - Add consultation remarks or relevant information.
    - Update appointment status as Confirmed, Completed, or Cancelled.

6. Patient Communication

    - Communicate with patients regarding appointment-related information.
    - Provide necessary instructions or updates related to the consultation.

Administrator

- Role: Manage the entire Book a Doctor platform, supervise users and doctors, manage appointments, and ensure the efficient and secure operation of the system.
- Flow

    1. System Administration

        - Monitor platform activities, user registrations, doctor accounts, and appointment statistics.
        - Ensure the smooth functioning of the Book a Doctor system.

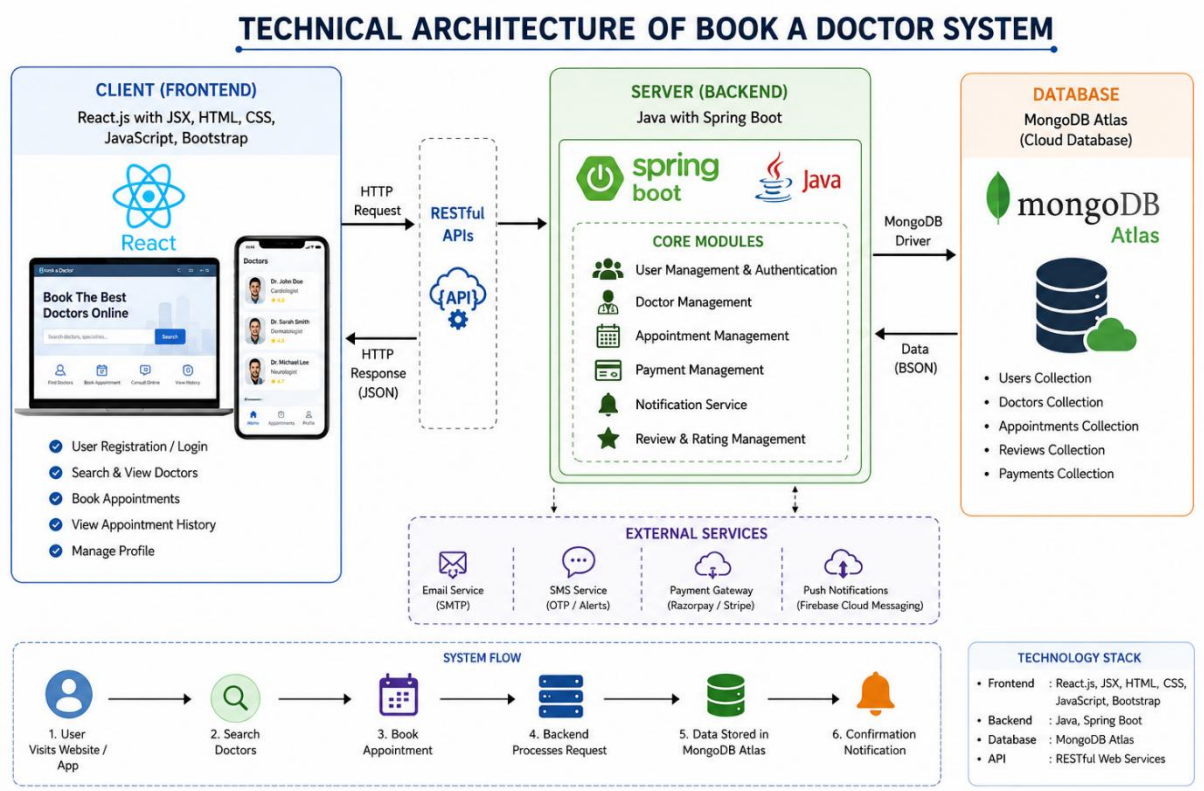
    2. Doctor Verification and Management

        - Verify and approve doctor registrations.
        - Manage doctor profiles, specializations, availability, and account access.

3. Appointment Management
    - Monitor scheduled, completed, cancelled, and pending appointments.
    - Manage appointment-related issues and resolve conflicts when required.
4. User Management
    - Manage patient accounts and user information.
    - Control access permissions and system roles.
5. Performance Monitoring
    - Monitor doctor activities and appointment statistics.
    - Generate reports and analytics related to users, doctors, and appointments.
6. Platform Security and Maintenance
    - Enforce authentication, authorization, and data protection measures.
    - Monitor system activities and maintain the security of patient and doctor information.
    - Implement system improvements and resolve operational issues.

MVC Pattern

Description



MVC Architecture

The **Book a Doctor** system follows the **Model-View-Controller (MVC)** architectural pattern. MVC divides the application into three main components: **Model, View, and Controller**. This separation makes the application organized, maintainable, scalable, and easier to develop.

1. Model Layer (Data Management)

The **Model** layer manages the application's data and database operations. It defines the structure of entities such as **Users, Doctors, Appointments, Medical Documents, and Administrators**.

The Model layer is responsible for creating, storing, retrieving, updating, and deleting data from the **MongoDB database**. It also helps maintain data consistency and integrity throughout the application.

2. Controller Layer (Business Logic)

The **Controller** layer handles requests received from patients, doctors, and administrators. It acts as a bridge between the **View and Model** layers and contains the main business logic of the application.

The Controller handles operations such as **user registration and login, doctor management, appointment booking, appointment status updates, document management, and user authentication**. After processing a request, it communicates with the Model to access or update database information and sends the appropriate response back to the View.

3. View Layer (User Interface)

The **View** layer represents the user-facing portion of the Book a Doctor application. It is developed using **React.js** and provides an interactive interface for patients, doctors, and administrators.

The View includes pages and components such as **Home Page, Registration, Login, Doctor Listing, Doctor Profile, Appointment Booking, Patient Dashboard, Doctor Dashboard, Admin Dashboard, and Profile Management**.

The View collects user input and displays the information received from the Controller in a simple and user-friendly format.

Advantages of MVC Architecture in This Project

1. Modular Design:

Each component has a specific responsibility, making the Book a Doctor application easier to understand, develop, and maintain.

2. Easy Maintenance:

Changes made to the user interface, business logic, or database structure can be managed independently, reducing development complexity.

3. **Scalability:**

   New features such as online payments, video consultations, prescriptions, reviews, and notifications can be added without significantly affecting the existing architecture.

4. Code Reusability:

   Models and controllers can be reused across different modules, reducing duplicate code and improving development efficiency.

5. Improved Testing:

   The Model, Controller, and View components can be tested independently, making it easier to identify and fix errors.

6. Team Collaboration:

   Developers can work simultaneously on the frontend interface, backend business logic, and database components, improving productivity and reducing development time.

CLIENT SETUP (Installing React Application)

Description

Setting Up the React Frontend

The frontend of the Book a Doctor application is developed using React.js with Vite. React provides a dynamic and interactive user interface, while Vite is used to create and run the frontend development environment efficiently.

Steps to Set Up the React Client

1. Open the Project Folder

Open the Book a Doctor project folder in Visual Studio Code and launch the integrated terminal.

2. Create the React Project

Create the React frontend using Vite by running the following command:

npm create vite@latest client -- --template react

3. Select the Required Options

When prompted, select:

- Framework: React
- Variant: JavaScript

4. Move to the Client Directory

Navigate to the newly created frontend directory:

cd client

5. Install Required Dependencies

Install the required React packages and project dependencies:

npm install

For the Book a Doctor application, additional packages such as Axios and React Router can be installed using:

npm install axios react-router-dom

6. Start the Development Server

Run the following command to start the React development server:

npm run dev

7. Open the Application

After the server starts successfully, Vite displays the local development URL in the terminal. Open the URL in a web browser:

http://localhost:5173

The Book a Doctor frontend will then be available in the browser for development and testing.

Expected Result

The React frontend is successfully configured and connected to the project structure. The client application provides the user interface for features such as Home, Login, Registration, Doctor Listing, Doctor Profile, Appointment Booking, Patient Dashboard, Doctor Dashboard, and Profile Management.

SERVER SETUP (npm init)

Backend Server Configuration

The backend of the **Book a Doctor** application is developed using **Node.js and Express.js**. It provides REST APIs for user authentication, doctor management, appointment booking, and communication with the MongoDB database.

Steps to Set Up the Server

1. Open the Server Folder

Open the **server** folder of the Book a Doctor project in **Visual Studio Code** and launch the integrated terminal.

2. Initialize the Node.js Project

Initialize a new Node.js project by executing the following command:

npm init -y

3. Verify the Project Configuration

The command automatically generates a **package.json** file containing the default project configuration and dependency information.

4. Install Required Backend Packages

Install Express, MongoDB/Mongoose, CORS, dotenv, and other required dependencies:

npm install express mongoose cors dotenv

For development, install Nodemon:

npm install --save-dev nodemon

5. Create the Main Server File

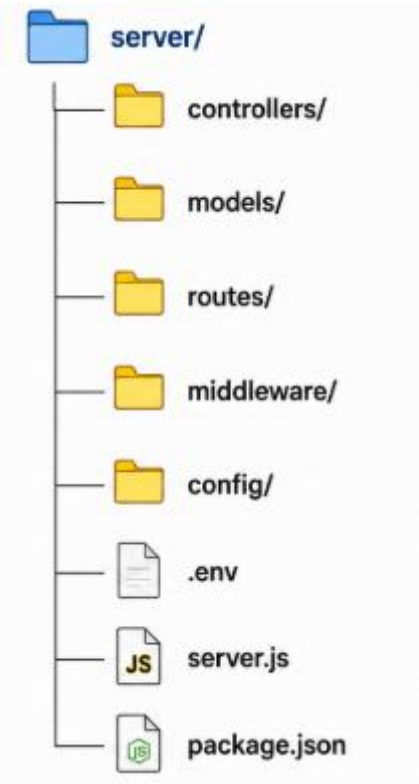
Create the main backend entry file:

server.js

This file is responsible for configuring Express, connecting the application to MongoDB, defining middleware, and starting the backend server.

6. Organize the Backend Project Structure

Create the following directories:



7. Purpose of Each Folder

- **models** – Contains MongoDB/Mongoose schemas for entities such as Users, Doctors, Appointments, and Medical Documents.
- **controllers** – Contains the application logic for handling registration, login, doctor management, appointment booking, and other operations.
- **routes** – Defines REST API endpoints and connects them to the appropriate controllers.
- **middleware** – Contains middleware used for authentication, authorization, error handling, and request processing.
- **config** – Contains configuration files such as database connection settings.
- **.env** – Stores environment variables such as the MongoDB connection string and server configuration.

8. Configure the Server

The Express server is configured to accept requests from the React frontend, process API requests, and communicate with MongoDB.

9. Start the Backend Server

Run the following command to start the server:

node server.js

If Nodemon is configured in package.json, the server can be started using:

npm run dev

The backend server will typically run at:

http://localhost:5000

Expected Result

After completing these steps, the basic backend structure of the **Book a Doctor** application is ready. The server can now provide REST APIs for **user registration and login, doctor management, appointment booking, appointment status updates, medical document management, and communication with the MongoDB database**.

DEVELOPMENT AND EXPLANATION

Project Setup

The development process begins by creating the **Book a Doctor** project with separate frontend and backend environments. The frontend is developed using **React.js with Vite**,

while the backend is developed using **Node.js and Express.js**. Required packages such as Express, Mongoose, CORS, Axios, dotenv, and other dependencies are installed to support the application.

Express Server Creation

An **Express.js** server is configured to handle requests from the React frontend and provide REST APIs. Middleware such as **JSON parsing, CORS, and environment configuration** is added to process request data, enable communication between the frontend and backend, and manage application settings.

Route Development

Separate route files are created for different modules of the Book a Doctor system, including **authentication, doctors, users, and appointments**. These routes define the available API endpoints and direct incoming requests to the appropriate controllers.

Controller Implementation

Controllers are developed to manage the application's business logic. They receive requests from the routes, validate the input, perform the required operations, interact with database models, and return appropriate responses to the React frontend.

Database Model Creation

Mongoose schemas and models are created for important entities such as **Users, Doctors, Appointments, and Medical Documents**. These models define the structure of the data and provide an organized way to create, retrieve, update, and delete information in the **MongoDB database**.

Doctor Management

The doctor module allows patients to browse available doctors and view information such as **name, specialization, experience, availability, and profile details**. Administrators can manage doctor accounts and verify doctor information before making the profiles available to patients.

Appointment Management

The appointment module allows patients to select a doctor, choose an available date and time slot, and book an appointment. Patients can view their upcoming and previous appointments, while doctors can view scheduled appointments and update their status as **Pending, Confirmed, Completed, or Cancelled**.

User Authentication

Authentication is implemented to securely manage patient, doctor, and administrator accounts. Users can register and log in using their credentials, while protected features and API routes are accessible only to authorized users according to their role.

Medical Document Management

The system allows patients to upload relevant medical reports and documents associated with their appointments. Doctors can access the required documents to understand the patient's information and support the consultation process.

Admin Operations

Admin-specific features are developed to manage **patients, doctors, appointments, and system activities**. Administrators can verify doctor accounts, manage user access, monitor appointments, and maintain the overall operation of the platform.

Notification and Status Updates

The system provides appointment-related updates to users, including **appointment confirmation, status changes, cancellation, and completion**. This helps patients and doctors stay informed about their scheduled consultations.

Error Handling

Error-handling mechanisms are implemented to manage invalid requests, authentication errors, database errors, and other application issues. Appropriate HTTP status codes and meaningful error messages are returned to improve system reliability and make debugging easier.

System Testing

All major modules are tested to ensure that the application functions correctly. **Registration, login, doctor search, appointment booking, appointment status updates, document management, database operations, and role-based access** are verified for accuracy, reliability, and proper performance.

CONFIGURE MONGODB

Install Mongoose

Mongoose is used in the Book a Doctor backend to connect the **Node.js/Express server** with the **MongoDB database**. It provides a structured way to define schemas, create models, and perform database operations.

Run the following command in the **server terminal**:

npm install mongoose

Create MongoDB Connection

After installing Mongoose, create a database connection in the backend using the MongoDB connection string. The connection details can be stored securely in the .env file.

Example:

MONGO_URI=your_mongodb_connection_string

The server uses this connection string to establish a connection with **MongoDB Atlas**.

Connect the Application to MongoDB

Mongoose is imported into the backend and used to establish the database connection. Once the connection is successful, the application can store and retrieve information such as:

- Users / Patients
- Doctors
- Appointments
- Medical Documents

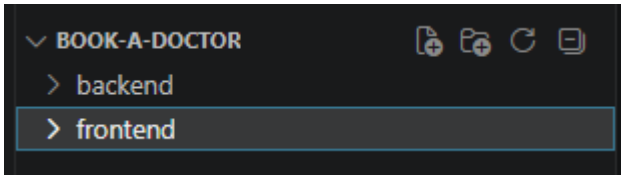
Database Operations

Mongoose models are used to perform common database operations such as:

- **Create** – Register users, doctors, and appointments.
- **Read** – Retrieve doctor profiles and appointment details.
- **Update** – Update profiles and appointment statuses.
- **Delete** – Remove records when required.

After the MongoDB connection is established successfully, the **Book a Doctor** backend is ready to store and manage application data securely.

CREATING FOLDER STRUCTURE



PROJECT DIRECTORY STRUCTURE

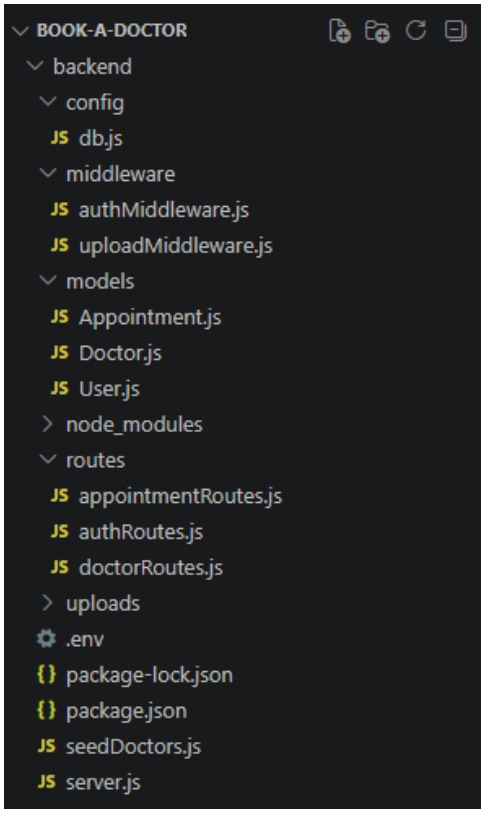
The **Book a Doctor** project is organized into a well-structured directory layout that separates the **frontend and backend components**, making the application easier to develop, maintain, test, and scale.

The **frontend** folder contains the client-side application developed using **React.js with Vite**. It includes pages, reusable components, assets, CSS files, routing configuration, and other files required to create the user interface for patients, doctors, and administrators.

The **backend** folder contains the server-side application developed using **Node.js and Express.js**. It includes REST APIs, controllers, database models, routes, middleware, configuration files, environment variables, and the main server file. The backend communicates with the **MongoDB database** to store and manage application data.

BACKEND FOLDER STRUCTURE

The **backend** handles the application's business logic and REST API operations. It manages **authentication, doctor information, appointment booking, user management, database operations, and other server-side functionality**.



The **Backend** folder contains the complete **server-side implementation** of the **Book a Doctor** system. It is responsible for processing client requests, implementing business logic, managing authentication, handling doctor and appointment operations, and communicating with the MongoDB database.

The **config** directory contains configuration-related files used for establishing and managing the database connection. The application uses **MongoDB Atlas** as the database and **Mongoose** for schema-based data modeling and database operations.

The **middleware** folder contains middleware functions used for request processing, authentication, authorization, validation, and error handling. These middleware functions help protect restricted resources and ensure that only authorized users can access specific features.

The **models** directory contains the **Mongoose schemas and models** used to represent the main entities of the Book a Doctor system. These include **User, Doctor, Appointment, and Medical Document** models. Each model defines the structure of the corresponding data stored in MongoDB.

The **routes** folder contains the **RESTful API endpoints** for different modules of the application. These routes handle functionalities such as **user registration and login, doctor management, appointment booking, appointment status updates, and user profile management**. Routes receive requests from the React frontend and forward them to the appropriate controllers.

The **controllers** folder contains the main **business logic** of the application. Controllers process incoming requests, validate data, interact with Mongoose models, perform required operations, and return appropriate responses to the frontend.

The **server.js** file acts as the **main entry point** of the backend application. It initializes the Express.js server, loads environment variables, configures middleware such as CORS and JSON parsing, establishes the MongoDB connection, registers API routes, and starts the server.

The **package.json** file contains project information, installed dependencies, and scripts required to run the backend. The **.env** file stores sensitive configuration values such as the MongoDB connection string and server-related environment variables, helping keep confidential information separate from the source code.

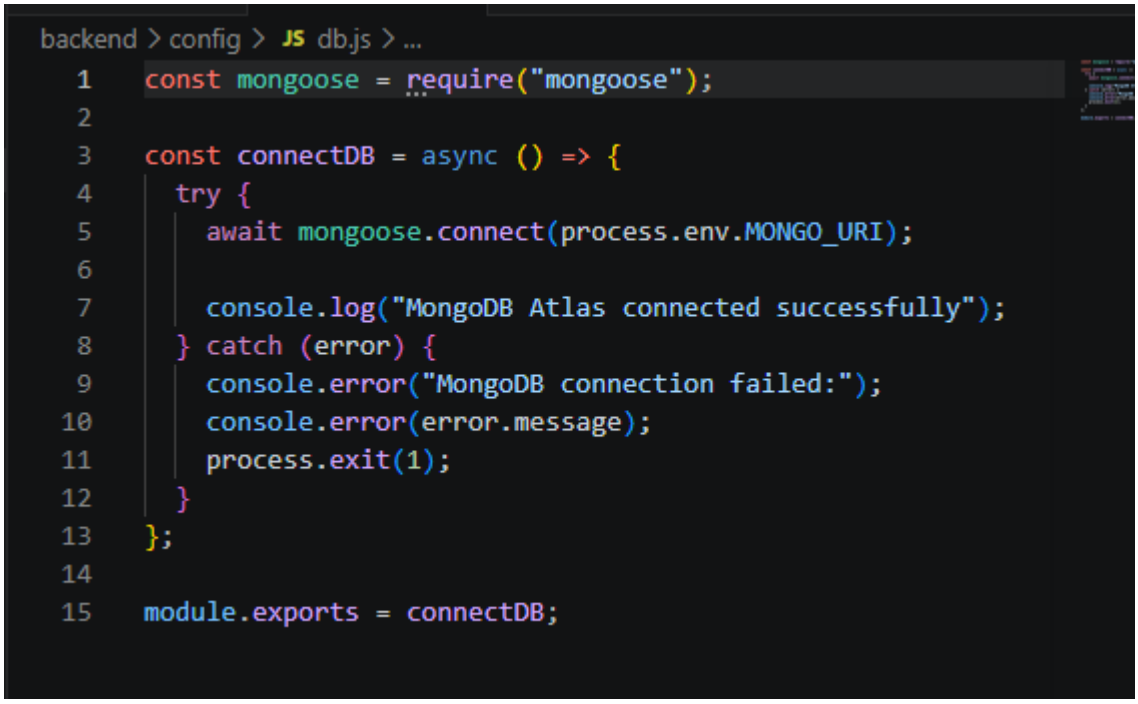
CREATE DATABASE CONNECTION

The Book a Doctor application uses **MongoDB Atlas** as its database. **Mongoose** is used to establish the connection between the Node.js/Express backend and MongoDB and to perform database operations.

The MongoDB connection string is stored securely in the .env file using an environment variable such as:

MONGO_URI=your_mongodb_connection_string

The database connection is initialized when the backend server starts. Once the connection is established successfully, the application can store and retrieve information related to **users, doctors, appointments, and medical documents**.



SCHEMA AND MODELS

The **Book a Doctor** application uses **Mongoose** to define schemas and models for storing and managing data in **MongoDB Atlas**. A schema specifies the structure, data types, and validation rules for each collection in the database. Mongoose models are then created from these schemas and are used by the backend to perform database operations.

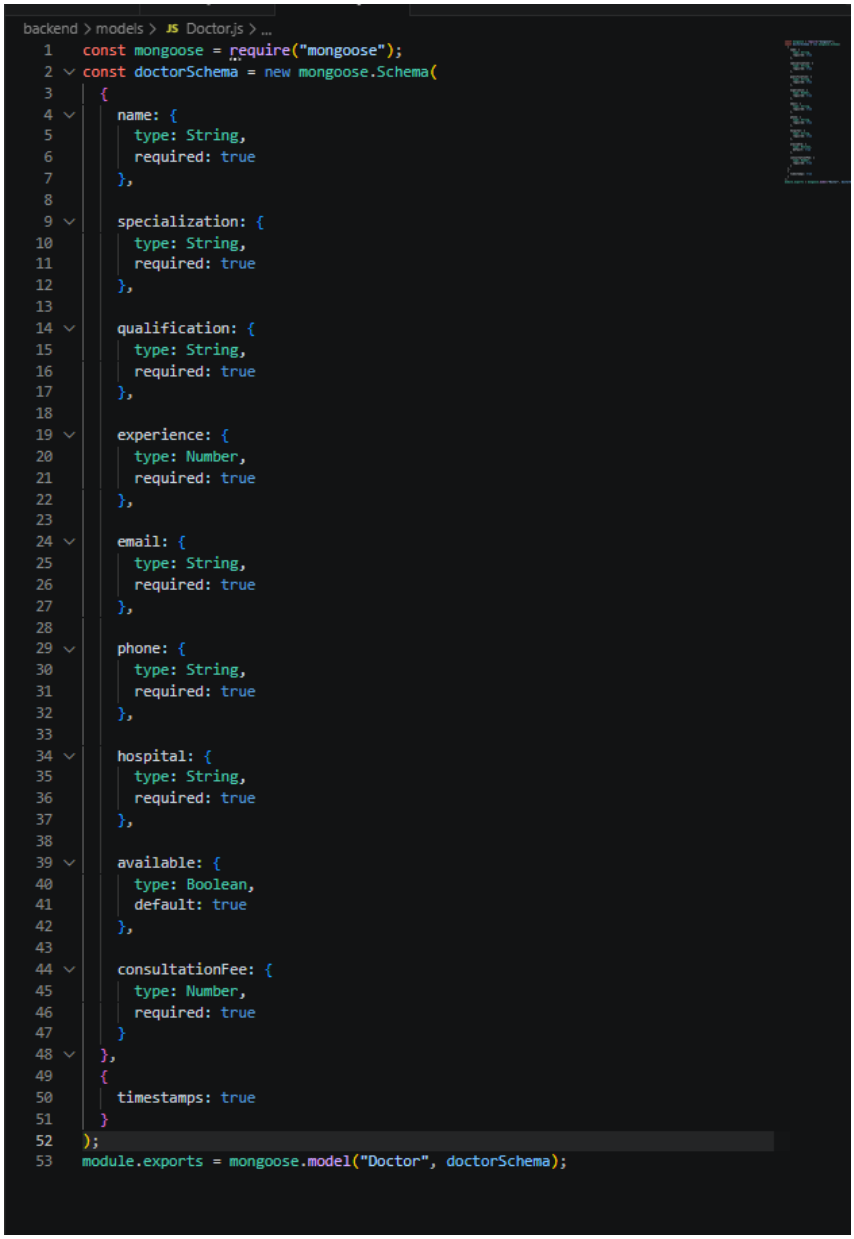
The main schemas used in the project are **User, Doctor, and Appointment**.

User Schema

The **User Schema** stores information about patients registered on the Book a Doctor platform. It contains details such as name, email, phone number, password, and user role.

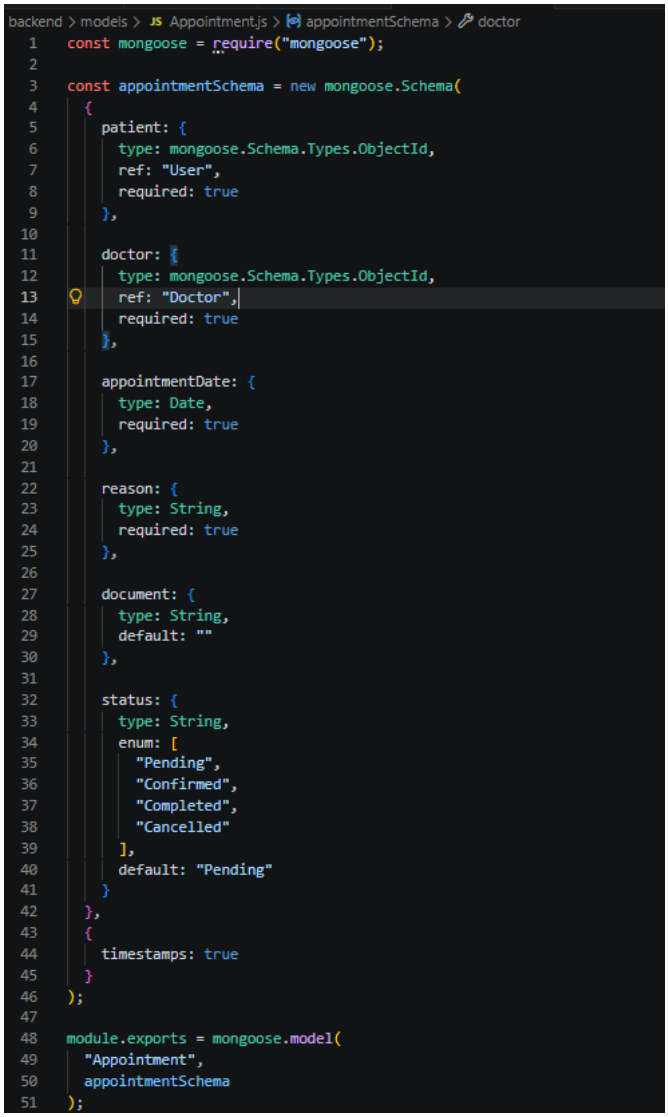
Doctor Schema

The **Doctor Schema** stores information about doctors available on the platform. It contains details such as doctor name, specialization, experience, qualification, availability, and profile information.



Appointment Schema

The **Appointment Schema** manages appointment information between patients and doctors. It stores details such as the patient, doctor, appointment date, appointment time, reason for consultation, and appointment status.



Schema and Model Working

• **Mongoose** provides schema-based data modeling for MongoDB Atlas.

• Each schema defines the structure and validation rules for a specific type of data.

• The **User model** manages patient account information.

• The **Doctor model** manages doctor profiles and availability.

• The **Appointment model** manages bookings between patients and doctors.

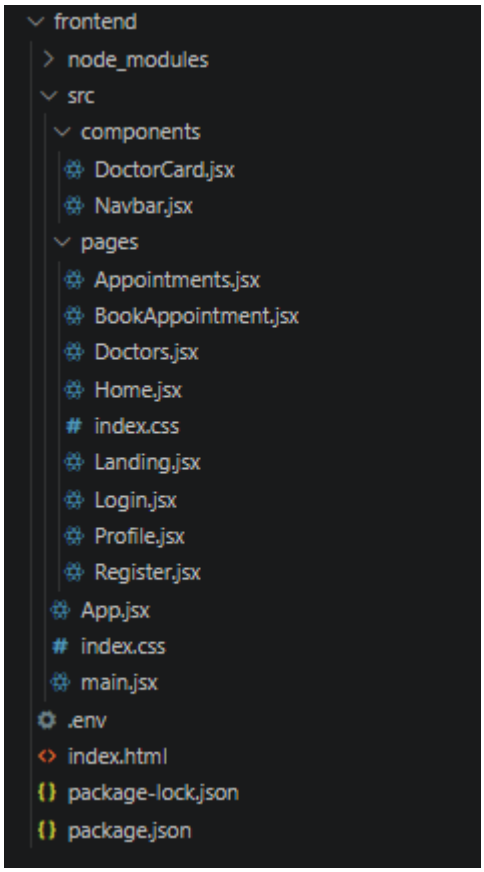
• The ObjectId references establish relationships between patients, doctors, and appointments.

• The models are imported into controllers to perform **Create, Read, Update, and Delete (CRUD)** operations.

• The data managed through these models is stored in the corresponding collections in **MongoDB Atlas**.

• This structured approach helps maintain data consistency, organization, and efficient database management.

FRONTEND FOLDER STRUCTURE



1. Setup React Application

The **frontend** folder contains the complete **client-side implementation** of the **Book a Doctor** application, developed using **React.js with Vite**. The application structure is organized inside the **src** directory to maintain clean, reusable, and scalable code.

The **components** folder contains reusable UI components used throughout the application. The **pages** folder contains screen-level components such as **Home, Login, Register, Doctors, Appointment Booking, Dashboard, and Profile** pages.

The **App.jsx** file manages the main application structure and routing between different pages, while **main.jsx** serves as the entry point that renders the React application in the browser. The **index.css** file manages global styling and provides a consistent visual appearance across the application.

The **package.json** file contains the required frontend dependencies and scripts used to run and manage the React application.

2. Design UI Components

The **UI components** are designed using **React.js** to provide a clean, responsive, and user-friendly interface for the Book a Doctor system. Reusable components are created for common elements such as the **Navbar, Doctor Cards, Appointment Forms, Buttons, Login Forms, Registration Forms, and Profile Sections**.

The navigation component provides access to important pages such as **Home, Doctors, Appointments, Login, Register, and Profile**. The application uses React Router to navigate between different pages without requiring a complete page reload.

The **Doctor Listing** interface allows patients to browse available doctors and view important information such as **doctor name, specialization, experience, and availability**. The **Appointment Booking** interface allows patients to select a doctor, date, and available time slot.

This component-based approach improves **code reusability, maintainability, scalability, and consistency** across the application.

3. Implement Frontend Logic

The frontend logic connects the React user interface with the backend REST APIs using **Axios**. Axios is used to send HTTP requests from the frontend to the Node.js and Express.js backend.

The frontend handles important operations such as:

• User registration and login.

• Fetching and displaying available doctors.

• Viewing doctor profile information.

• Booking appointments.

• Viewing upcoming and previous appointments.

• Updating and displaying appointment status.

• Managing user profile information.

• Uploading and accessing relevant medical documents when supported.

The **App.jsx** file manages application routing and connects the different pages of the system. React state and event handling are used to manage user interactions, form inputs, doctor information, appointment details, and API responses.

The frontend communicates with the backend through REST API endpoints, while the backend processes requests and interacts with **MongoDB Atlas**. This creates a clear separation between the **user interface, application logic, and database layer**, making the Book a Doctor application easier to maintain and expand.

PROJECT EXECUTION

Steps for Project Execution

Step 1: Open the Project

Open the **Book a Doctor** project folder in Visual Studio Code and launch the terminal.

```
cd Book-A-Doctor
```

Step 2: Install Frontend Dependencies

Navigate to the frontend folder:

```
cd frontend
```

Install all required frontend dependencies:

```
npm install
```

Start the React development server:

```
npm run dev
```

The frontend application will normally be available at:

http://localhost:5173

Step 3: Install Backend Dependencies

Open another terminal in Visual Studio Code and navigate to the server folder:

```
cd server
```

Install the required backend dependencies:

```
npm install
```

Step 4: Configure MongoDB Atlas

Create a .env file inside the **server** folder and add your MongoDB Atlas connection string:

```
MONGO_URI=your_mongodb_atlas_connection_string

PORT=5000
```

The MongoDB Atlas connection string is used by the backend to connect securely to the cloud database.

Step 5: Start the Backend Server

Start the Node.js/Express backend server:

```
node server.js
```

If your project has a development script configured with Nodemon, you can use:

```
npm run dev
```

The backend server will normally run at:

http://localhost:5000

Step 6: Run the Complete Application

After successfully starting both servers:

- **Frontend:** http://localhost:5173
- **Backend:** http://localhost:5000
- **Database:** MongoDB Atlas

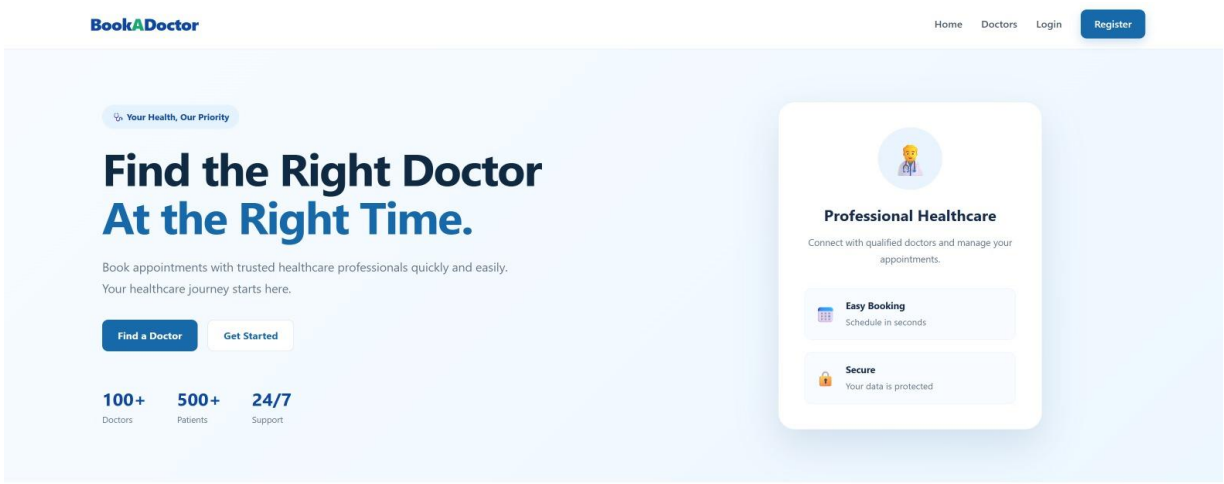
Open http://localhost:5173 in the browser to access the **Book a Doctor** application.

The React frontend communicates with the Express backend through REST APIs, while the backend communicates with **MongoDB Atlas** to store and retrieve users, doctors, appointments, and other application data.

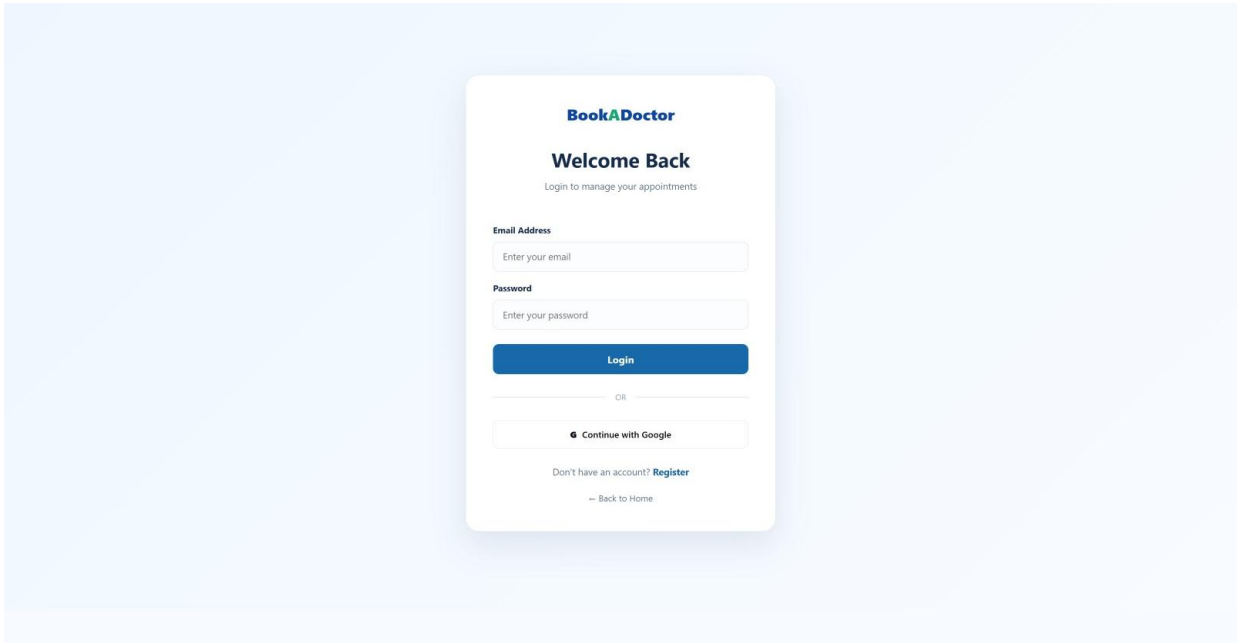
DEMO SCREENSHORTS

Output ScreenShots:

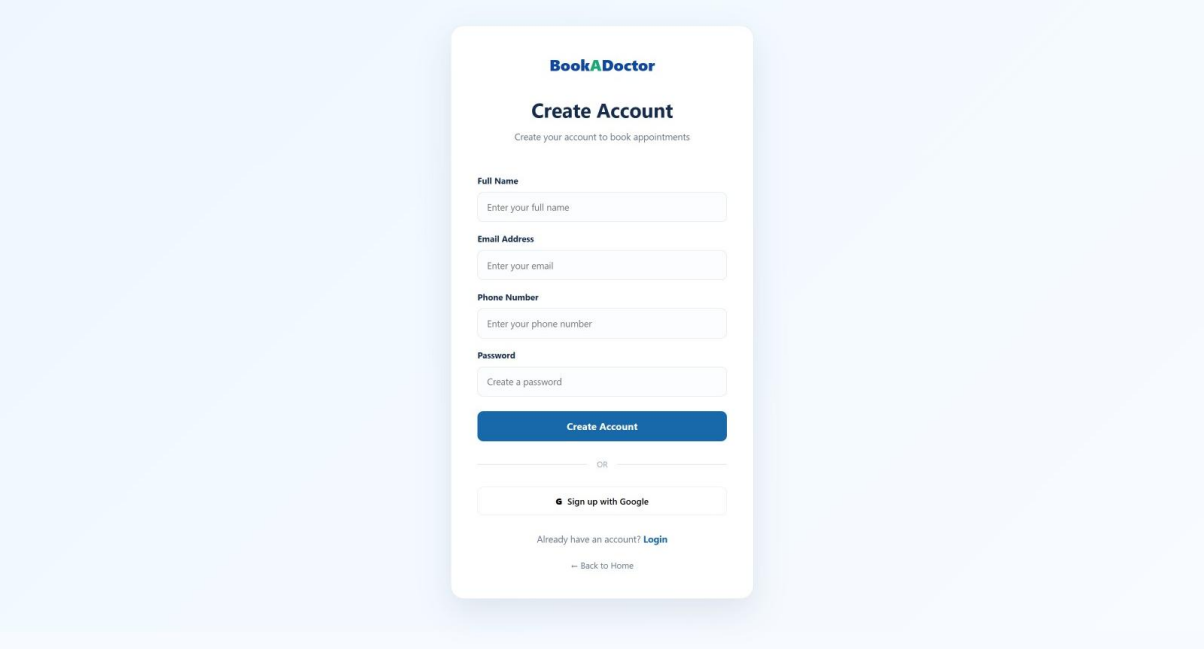
- **Landing Page:** Displays a welcoming introduction to the Book a Doctor application with options to Login, Register, or continue as a guest. It serves as the main entry point for patients and provides access to the doctor-booking services.



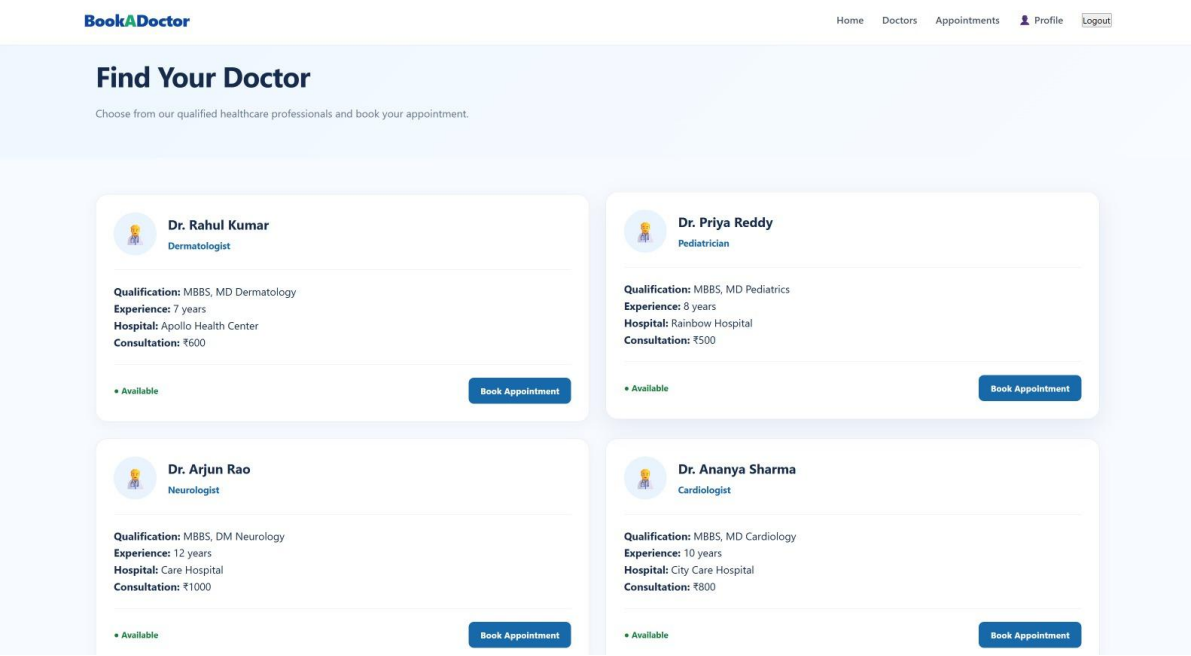
- **Login Page:** Allows registered patients, doctors, and administrators to securely log in using their credentials.



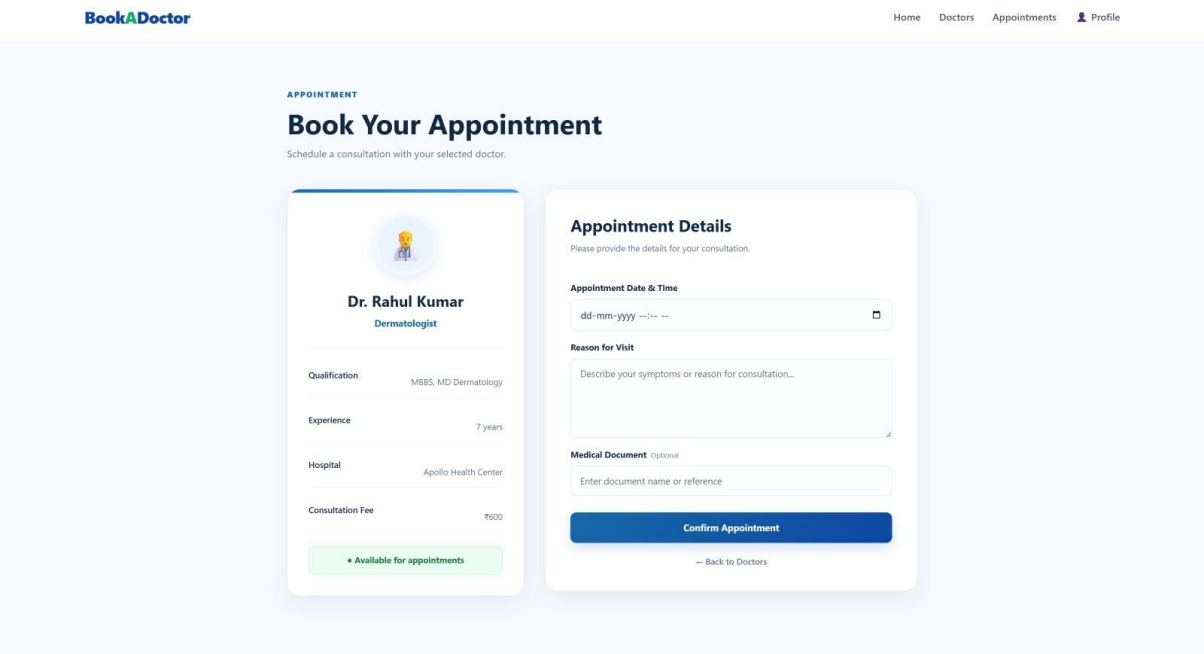
- **Registration Page:** Allows new patients to create an account by providing their required personal and contact information.



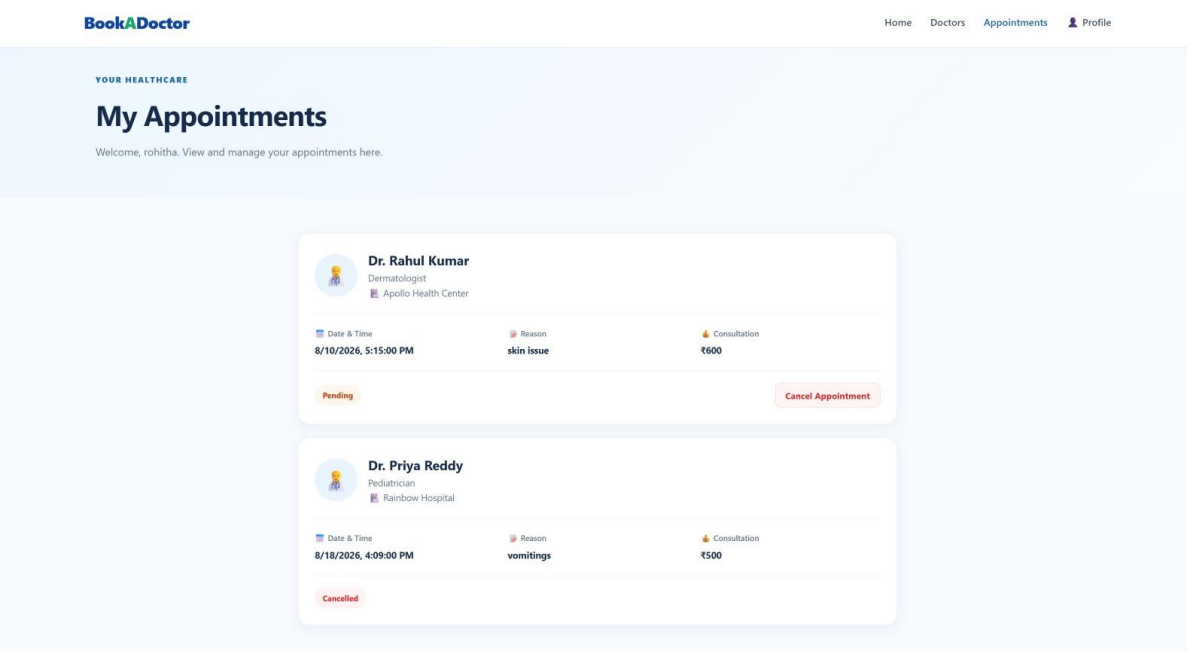
- **Doctors Page:** Displays available doctors along with their **specialization, experience, qualification, and availability**. Patients can browse and select a suitable doctor.



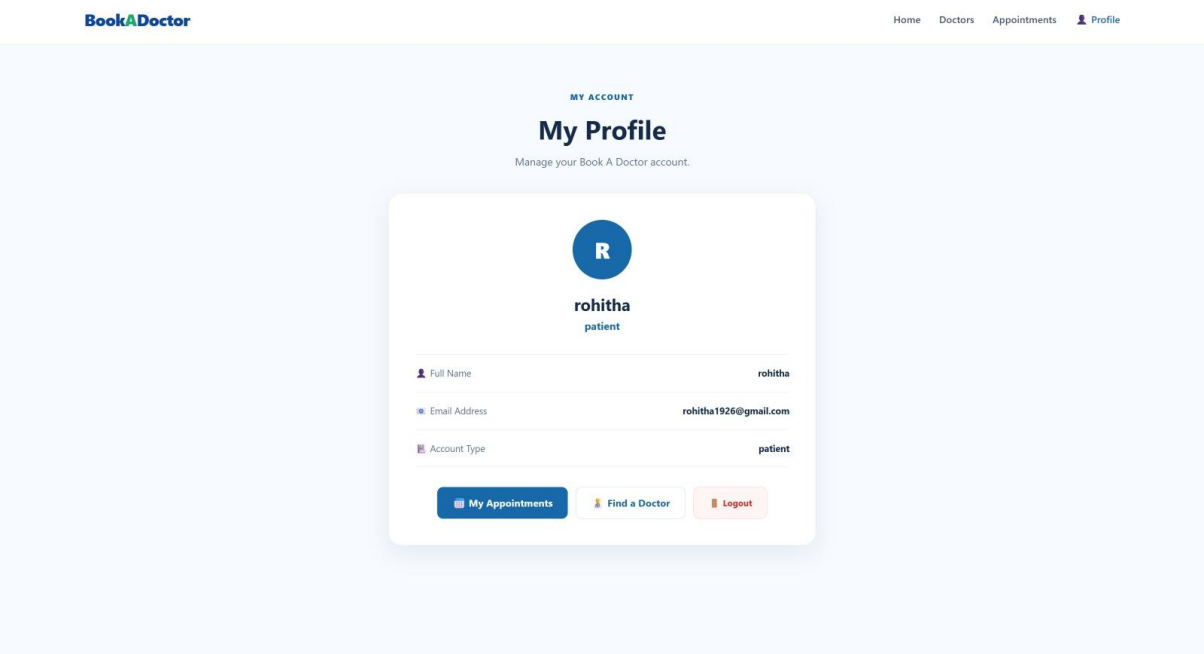
- **Doctor Profile & Appointment Booking Page:** Displays detailed doctor information and allows patients to select an available **date and time slot** and confirm an appointment.



- **My Appointments Page:** Allows patients to view and manage their **upcoming and previous appointments**, including doctor details, appointment date, time, and current appointment status.



- **Profile Page:** Allows users to view and update their personal information and account details.



DEMO LINK

http://drive.google.com/file/d/1dwC2lXVSPk0FeiWsDCCOvF7qbRVJagCx/view?pli=1